Pattern-Based Power Planning Examples

Use the following pattern-based power planning examples as a guideline when creating a power plan for your design. Copy and paste these examples, then modify them as needed.

Table of Contents

Rings

PG Rings This example creates a simple pattern of four PG rings for the VDD and VSS nets.

PG Ring With Corner Bridging This example creates a pattern of four PG rings with corner bridging for two nets: VDD and VSS.

Mesh

Simple Mesh This example creates a simple PG mesh for two layers (M5 and M6) and four nets (VDD1, VDD2, VSS1, VSS2).

Simple Mesh with Parameters This example creates a simple PG mesh with parameters for two layers (M5 and M6) and four nets (VDD1, VDD2, VSS1, VSS2).

Simple Mesh with Distributed Spacing This example creates a simple PG mesh for two layers (M5 and M6) and four nets (VDD1, VDD2, VSS1, VSS2).

Mesh/Std Rail with Via Rule This example creates a simple PG mesh for five layers (M5, M8, M9, M10, and M11).

Multiple Mesh with Via Rule Between Mesh This example creates two PG meshes.

Mesh with Via Rule on Tag of Existing Wires This example creates two PG meshes.

Mesh with Shifted Stack Vias This example creates a simple PG mesh for four layers (K4, K3, M3 and M2).

Mesh as Via Ladders for Switch Cells This example creates an M6 to M2 mesh for each switch cell.

Straps

M4 Single Layer PG Straps With Distributed Spacing This example creates single-layer metal straps on M4.

Aligning Straps to Specific Mask Tracks. This example creates single-layer metal straps on M2.

Vias

Vias Between Parallel M4 Mesh and M2 Rail This example creates single-layer metal straps on M4 and standard cell rails on M1 and M2.

Vertically Aligned Vias Between Parallel M4 Mesh and M2 Rail This example creates single-layer metal straps on M4 and standard cell rails on M1 and M2.

Vias Between Parallel M3 Mesh and M1 Rail on Special Cells This example creates vias between existing horizontal M3 shapes and M1 rails only on specific diode cells.

PG Straps and Vias This example avoids the pattern-based power planning commands to manually create PG straps and insert vias.

Vertically Aligned Vias This example creates vertically aligned vias between parallel wires M4 and M2.

Multiple ContactCodes to Repair Via DRC This example creates vias between M6 and M7, with contactCodes VIA67_BW114, VIA67_BW76, VIA67_BW38_UW38, and VIA67_BW21.

Vias between Parallel Wires and Rails Considering Signal Routes and Standard Cells Pins This example creates vias between parallel wires M3 and rails M1.

Custom Vias at Specified Locations This example creates custom vias with 8 1x1 cuts at specified locations.

Stapling Vias This example inserts stapling vias with dimension 10x1 between the adjacent layers M1 and M2 in areas where the layers overlap.

Stapling Vias Between Non-Adjacent Layers This example inserts stapling vias between non-adjacent layers M3 and M1 in areas where the layers overlap.

Rails and Pins

PG Standard Cell Rails This example inserts standard cells rails for nets VDD and VSS on layer M1.

PG Mesh and Standard Cell Rails This example creates a mesh on vertical layers M4 and M6 and horizontal layer M5, and standard cells rails for nets VDD and VSS on layer M1.

Macro Pin Connection This example creates PG pin connections for hard macros.

Pad Pin Connection This example creates PG pin connections for I/O pad IO_E1VCC_0.

Pad Pin Connection to Rings This example creates PG pin connection to ring for I/O pad I_VSSIO_L01.

Tap Cell Pin Connection This example creates PG pin connections for tap cells.

Alignment Straps for Switch Cells This example creates vertical PG alignment straps to connect VDD pins on layer M2 of switch cells with master HEADBUFTIE42Q_D3.

Channel Straps This example creates straps in channels between two macros, two placement blockages, and one macro and one placement blockage.

Composite Patterns

Composite Pattern from Three Wire Patterns This example creates a composite pattern that contains three individual pattern descriptions.

Composite Pattern with PG Region and Offset Defined in Wire Pattern This example creates a composite pattern that contains three individual pattern descriptions.

Composite Pattern with Custom Vias Between Parallel Wires This example creates a composite pattern that contains two wire patterns on layers M7 and M9.

Composite Pattern with Custom Via This example defines custom via defs and creates the corresponding custom vias in the middle of two vertical wires.

Composite Pattern with Tcl Variables This example creates a composite pattern that contains four individual pattern descriptions.

Composite Pattern for Advanced Nodes This example create a power grid that connects to M0 rails for an advanced technology node.

Other Examples

Center-Aligned Vias This example creates horizontal power straps on layer M3 and vertical power straps on layer M8.

Bridging Straps This example creates bridging straps on layers M3 and M5 to connect vertical power straps on layer M6.

Via Bridging This example creates a complex via bridging structure to connect the vertical VDD strap on layer M8 with the horizontal VDD strap on layer M3.

Checkerboard Via Pattern This example creates a mesh pattern using only VDD nets, then inserts vias between alternating intersections of layers M4 and M5 to form a checkerboard pattern of vias.

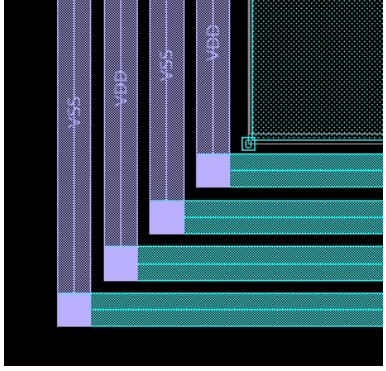
Tapering Straps This example creates a power structure with narrowed power straps at the center of the die.

PG Rings

Simple PG Ring *Go to top*

This example creates a simple pattern of four PG rings for the VDD and VSS nets.

```
create_pg_ring_pattern ring_pattern -horizontal_layer M7 \
```



```
   -horizontal_width {5} -horizontal_spacing {2} \
   -vertical_layer M8 -vertical_width {5} -vertical_spacing {2}

set_pg_strategy core_ring \
   -pattern {{name: ring_pattern} \
   {nets: {VDD VSS VDD VSS}} {offset: {3 3}}} -core

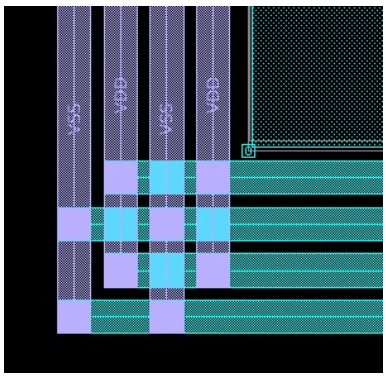
compile_pg -strategies core_ring
```

Commands used:
create_pg_ring_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

PG Ring With Corner Bridging *Go to top*

This example creates a pattern of four PG rings with corner bridging for two nets: VDD and VSS. A corner bridge is a configuration where the individual power straps that make up the ring are connected to other power straps in the ring that share the same net.



```
create_pg_ring_pattern ring_pattern -horizontal_layer M7 \
   -horizontal_width {5} -horizontal_spacing {2} \
   -vertical_layer M8 -vertical_width {5} -vertical_spacing {2} \
                      -corner_bridge true

set_pg_strategy core_ring \
   -pattern {{name: ring_pattern} \
   {nets: {VDD VSS VDD VSS}} {offset: {3 3}}} -core

compile_pg -strategies core_ring
```

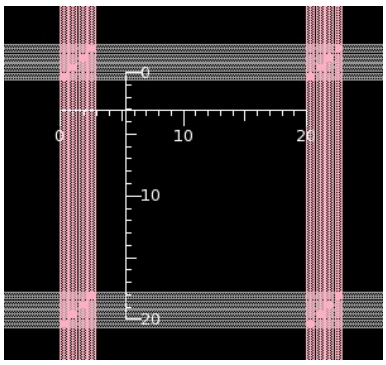
Commands used:
create_pg_ring_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

PG Meshes

Simple Mesh *Go to top*

This example creates a simple PG mesh for two layers (M5 and M6) and four nets (VDD1, VDD2, VSS1, VSS2).




```
create_pg_mesh_pattern mesh_pattern \
   -layers {{{vertical_layer: M6} {width: 0.6}\
             {pitch: 20} {offset: 20}}\
            {{horizontal_layer: M5} {width: 0.6}\
             {pitch: 20} {offset: 20}}}

set_pg_strategy M5M6_mesh \
   -pattern {{name: mesh_pattern} \
             {nets: VDD1 VSS1 VDD2 VSS2}} -core

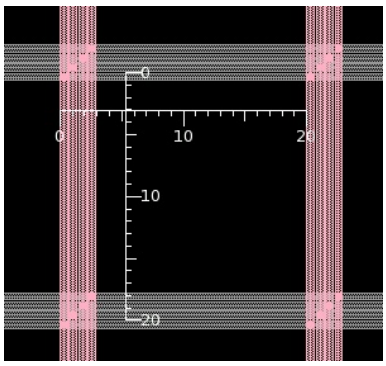
compile_pg -strategies M5M6_mesh
```

Commands used:
create_pg_mesh_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Simple Mesh with Parameters *Go to top*

This example creates a simple PG mesh with parameters for two layers (M5 and M6) and four nets (VDD1, VDD2, VSS1, VSS2). Five Tcl variables are used as parameter values in strategy M5M6_mesh.




```
create_pg_mesh_pattern mesh_pattern \
   -parameters {vlayer hlayer w p o} \
   -layers {{{vertical_layer: @vlayer} {width: @w}\
             {pitch: @p} {offset: @o}}\
            {{horizontal_layer: @hlayer} {width: @w}\
             {pitch: @p} {offset: @o}}}

set layer1 M6
set layer2 M5
set width 0.6
set pitch 20
set offset 20

set_pg_strategy M5M6_mesh \
   -pattern {{name: mesh_pattern} \
             {nets: VDD1 VSS1 VDD2 VSS2} \
             {parameters: $layer1 $layer2 $width $pitch $offset}} -core

compile_pg -strategies M5M6_mesh
```

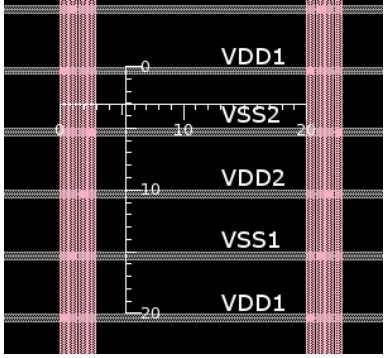
Commands used:
create_pg_mesh_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Simple Mesh with Distributed Spacing *Go to top*

This example creates a simple PG mesh for two layers (M5 and M6) and four nets (VDD1, VDD2, VSS1, VSS2). The four nets on the horizontal layer M5 are evenly distributed with the `{spacing:interleaving}` directive and maintain a pitch of 20 between shapes of the same net.

```
create_pg_mesh_pattern mesh_pattern \
   -layers {{{vertical_layer: M6} {width: 0.6}\
             {pitch: 20} {offset: 20}}\
            {{horizontal_layer: M5} {width: 0.6}\
```




```
            {pitch: 20} {spacing: interleaving}}}

set_pg_strategy M5M6_mesh \
   -pattern {{name: mesh_pattern} \
            {nets: VDD1 VSS1 VDD2 VSS2}} -core

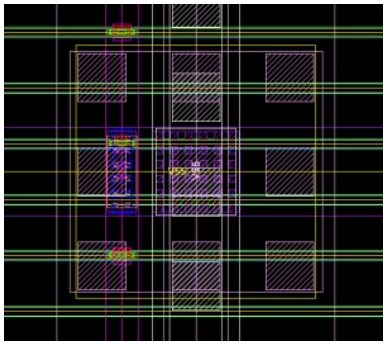
compile_pg -strategies M5M6_mesh
```

Commands used:
create_pg_mesh_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Mesh/Std Rail with Via Rule *Go to top*

This example creates a simple PG mesh for five layers (M5, M8, M9, M10, and M11). Via rules among the layers are defined in the mesh_pattern. Std rails are also created for layer M1 and M2. Strategy Via rule rail_via_rule is defined to ensure no vias are created between std rails and existing wires. Strategy Via rule mesh_via_rule is defined to create vias between mesh pattern on layer M5 and existing std rail on M2.



```
create_pg_std_cell_conn_pattern rail_pat -layers {M1 M2}

set_pg_strategy rail_strategy -core -pattern {{name: rail_pat} {nets: {VDD VSS}}}

set_pg_strategy_via_rule rail_via_rule -via_rule \
                         {{intersection: all} {via_master: NIL}}

compile_pg -strategies {rail_strategy} -via_rule rail_via_rule

create_pg_mesh_pattern mesh_pattern -layers { \
      {{vertical_layer: M5} {width: 0.28} {spacing: interleaving} {pitch: 5.6}}  \
      {{horizontal_layer: M8} {width: 0.756} {spacing: interleaving} {pitch: 6.24}} \
      {{vertical_layer: M9} {width: 0.756} {spacing: interleaving} {pitch: 6.24}} \
      {{vertical_layer: M10} {width: 2.4} {spacing: interleaving} {pitch: 6.24}} \
      {{horizontal_layer: M11} {width: 2.4} {spacing: interleaving} {pitch: 6.24}} } \
      -via_rule { \
         {{layers: M5} {layers: M8} {via_master: default}} \
         {{layers: M8} {layers: M9} {via_master: default}} \
         {{layers: M9} {layers: M10} {via_master: default} {between_parallel:true}} \
         {{layers: M10} {layers: M11} {via_master: default}} \
         {{intersection: undefined} {via_master: NIL}} \
      }

set_pg_strategy mesh_strategy -core -pattern \
       {{name: mesh_pattern} {nets: {VDD VSS}}} -extension {{stop: design_boundary}}

set_pg_strategy_via_rule mesh_via_rule -via_rule { \
       {{{strategies: mesh_strategy} {layers: M5}} \
        {{existing: std_conn} {layers: M2}} {via_master: default}} \
       {{intersection: undefined} {via_master: NIL}} \
      }

compile_pg -strategies {mesh_strategy} -via_rule mesh_via_rule
```

Commands used:
create_pg_std_cell_conn_pattern, create_pg_mesh_pattern, set_pg_strategy, set_pg_strategy_via_rule, and compile_pg.

Copy to script editor

Multiple Mesh with Via Rule Between Mesh *Go to top*

This example creates two PG meshes. One is for layers M2 and M5, and another is for layers M6, M7, M8, and M9. For M2_M5 mesh, default vias are created between M2 and M5. For M6_M7_M8_M9 mesh, vias are created only between adjacent layers. Standard cell rails are created on layer M1. A strategy via rule named "strategy_via_rule" is created and used in compile_pg to ensure vias are created between only adjacent layers of wires among the two mesh and standard cells rails. Note that strategy via rule is needed for control between layer of mesh pattern and layer of other pattern because create_pg_mesh_pattern -via_rule option doesn't control it. The -via_rule option of create_pg_mesh_pattern is for controlling vias created for intersection of wires in the pattern.




```
create_pg_std_cell_conn_pattern PgPattern_m1 -layers {M1}

set_pg_strategy PgStrategy_m1 -core \
         -pattern {{name: PgPattern_m1} {nets: VDD VSS}}

create_pg_mesh_pattern PgPattern_m2_m5 -layers {
    {{horizontal_layer: M2} {width: 0.076 0.076} \
     {pitch: 0.96} {spacing: 0.404}} \
    {{vertical_layer: M5} {width: 0.56 0.56} \
     {pitch: 11.2} {spacing: 5.04}} \
  } -via_rule { \
    {{layers: M5} {layers: M2} {via_master: default}} \
    {{intersection: undefined} {via_master: NIL}} \
  }

set_pg_strategy PgStrategy_m2_m5 -core \
       -pattern {{name: PgPattern_m2_m5} {nets: VDD VSS}}

create_pg_mesh_pattern PgPattern_m6_m7_m8_m9 -layers { \
    {{horizontal_layer: M6} {width: 1.324 1.324} \
     {pitch: 10.584} {spacing: 3.968}} \
    {{vertical_layer: M7} {width: 1.324 1.324} \
     {pitch: 10.584} {spacing: 3.968}} \
    {{horizontal_layer: M8} {width: 3.5 3.5} \
     {pitch: 24.0} {spacing: 2.0}} \
    {{vertical_layer: M9} {width: 3.5 3.5} \
     {pitch: 24.0} {spacing: 2.0}} \
  } -via_rule {{intersection: adjacent} {via_master: default}}

set_pg_strategy PgStrategy_m6_m7_m8_m9 -core \
      -pattern {{name: PgPattern_m6_m7_m8_m9} {nets: VDD VSS}} \
      -extension {{stop: design_boundary_and_generate_pin}}
```

```
set_pg_strategy_via_rule strategy_via_rule  \
  -via_rule {{intersection: adjacent}{via_master: {default}}}

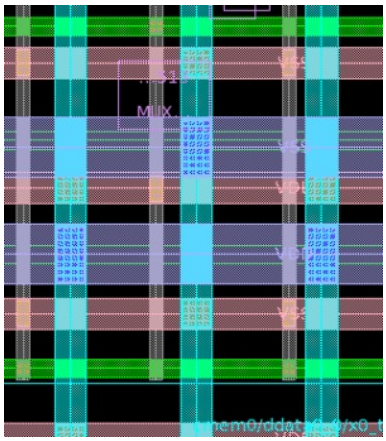
compile_pg -strategies {PgStrategy_m1 PgStrategy_m2_m5 PgStrategy_m6_m7_m8_m9} \
  -via_rule strategy_via_rule
```

Commands used:
create_pg_std_cell_conn_pattern, create_pg_mesh_pattern, set_pg_strategy, set_pg_strategy_via_rule, and compile_pg.

Copy to script editor

Mesh with Via Rule on Tag of Existing Wires *Go to top*

This example creates two PG meshes. One is for layers M4 and M5, and another is for layers M6, M7, and M8. For M4_M5 mesh, default vias are created between M4 and M5. After the mesh is created, the tag name "twire" is applied to M5 wires. For M6_M7_M8 mesh, vias are created only between adjacent layers. A strategy via rule named "strategy_via_rule" is created for intersection between shapes from strategy PgStrategy_m6_m7_m8 on layer M6 and the existing shapes with the tag name "twire". The via rule is used in compile_pg to ensure vias are created only for intersection between PgStrategy_m6_m7_m8 M6 wires and existing wires with tag "twire".



```
create_pg_mesh_pattern PgPattern_m4_m5 -layers {
    {{horizontal_layer: M4} {width: 0.76 0.76} \
     {pitch: 9.6} {spacing: 4.04} {trim: false}} \
    {{vertical_layer: M5} {width: 0.56 0.56} \
     {pitch: 11.2} {spacing: 5.04} {trim: false}} \
  } -via_rule { \
    {{layers: M5} {layers: M4} {via_master: default}} \
    {{intersection: undefined} {via_master: NIL}} \
  }

set_pg_strategy PgStrategy_m4_m5 -core \
       -pattern {{name: PgPattern_m4_m5} {nets: VDD VSS}}

compile_pg -strategies PgStrategy_m4_m5 -tag m4_m5

set_attribute [get_shapes -filter "layer_name==M5 && tag==m4_m5"] tag twire

create_pg_mesh_pattern PgPattern_m6_m7_m8 -layers { \
    {{horizontal_layer: M6} {width: 1.324 1.324} \
     {pitch: 10.584} {spacing: 3.968}} \
    {{vertical_layer: M7} {width: 1.324 1.324} \
     {pitch: 10.584} {spacing: 3.968}} \
    {{horizontal_layer: M8} {width: 2.5 2.5} \
     {pitch: 24.0} {spacing: 2.0}} \
  } -via_rule {{intersection: adjacent} {via_master: default}}

set_pg_strategy PgStrategy_m6_m7_m8 -core \
     -pattern {{name: PgPattern_m6_m7_m8} {nets: VDD VSS}} \
     -extension {{stop: design_boundary_and_generate_pin}}

set_pg_strategy_via_rule strategy_via_rule  \
  -via_rule { \
     {{{strategies: PgStrategy_m6_m7_m8}{layers: M6}} \
      {tags: {twire}}{via_master: {default}}} \
     {{intersection: undefined} {via_master: NIL}} \
   }

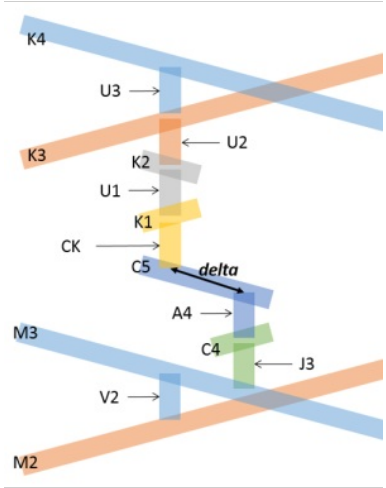
compile_pg -strategies {PgStrategy_m6_m7_m8} \
  -via_rule strategy_via_rule
```

Commands used:
create_pg_mesh_pattern, set_pg_strategy, set_pg_strategy_via_rule, and compile_pg.

Copy to script editor

Mesh with Shifted Stack Vias *Go to top*

This example creates a simple PG mesh for four layers (K4, K3, M3 and M2). Stacked vias between layers K3 and M3 are via U2, U1, CK, shifted A4, and shifted J3 (see the picture below). U2C, U1C, CKC, A4C and J3C are contact codes for layers U2, U1, CK, A4, and J3, respectively.




```
set delta 0.5
set_pg_via_master_rule A4V -contact_code A4C -offset {$delta 0}
set_pg_via_master_rule J3V -contact_code J3C -offset {$delta 0}

create_pg_mesh_pattern mesh \
           -parameters {w1 w2 w3 w4 f} \
           -layers { \
              {{horizontal_layer: K4}{width: @w1} \
                {spacing: interleaving} {pitch: 20} {offset: @f}} \
              {{vertical_layer: K3}{width: @w2} \
                {spacing: interleaving}{pitch: 20} {offset: @f}} \
              {{horizontal_layer: M3}{width: @w3} \
                {spacing: interleaving} {pitch: 20} {offset: @f}} \
              {{vertical_layer: M2}{width: @w4} \
                {spacing: interleaving}{pitch: 20} {offset: @f}} \
           } \
           -via_rule { \
                {{layers: K4}{layers: K3}{via_master: default}} \
                {{layers: K3}{layers: M3}{via_master: {U2C U1C CKC A4V J3V}} \
                {{layers: M3}{layers: M2}{via_master: default}} \
                {{intersection: undefined}{via_master: NIL}} \
           }

set_pg_strategy mesh_strategy -core -pattern \
       {{name: mesh_pattern} {nets: {VDD VSS}} {parameters: {5 4 3 2 10}}}

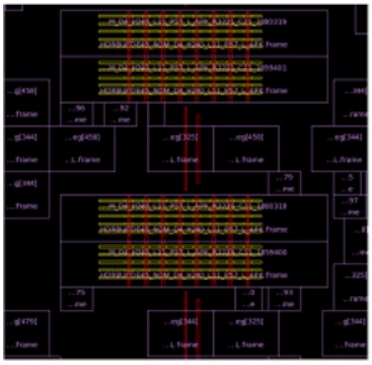
compile_pg -strategies {mesh_strategy}
```

Commands used:
set_pg_via_master_rule, create_pg_mesh_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Mesh as Via Ladders for Switch Cells *Go to top*

This example creates an M6 to M2 mesh for each switch cell. The mesh is used as via ladder for switch cells to reduce via resistance. In the example, a PG region is created for each switch cell, and a mesh with specified pattern is created for each created PG region.



```
set count 0
set sw_collection [get_cells -filter "is_power_switch==true"]
foreach_in_collection cell $sw_collection {
  create_pg_region switch_region_${count} -polygon [get_attribute -objects $cell -name boundary] \
      -expand_by_edge {{{side:1} {offset: -0.4}} {{side:3} {offset: -0.693}}}
  incr count
}

#create_pg_mesh_pattern for switch cells
create_pg_mesh_pattern pg_mesh_sw -layers { \
        {{horizontal_layer: M2} {width: 0.02} {pitch: 0.08} {offset: 0.1} \
          {trim: false} {track_alignment: track}} \
        {{vertical_layer: M3} {width: 0.024} {pitch: 0.176} {offset: 0.318} \
          {trim:false} {track_alignment: track}} \
        {{horizontal_layer: M4} {width: 0.038} {pitch: 0.228} {offset: 0.136} \
          {trim: false} {track_alignment: track}} \
        {{vertical_layer: M5} {width: 0.038} {pitch: 0.152} {offset: 0.17} \
          {trim:false} {track_alignment: track}} \
        {{horizontal_layer: M6} {width: 0.038} {pitch: 0.228} {offset: 0.136} \
          {trim:false} {track_alignment: track}}} \
        -via_rule {{intersection: adjacent} {via_master: default}}

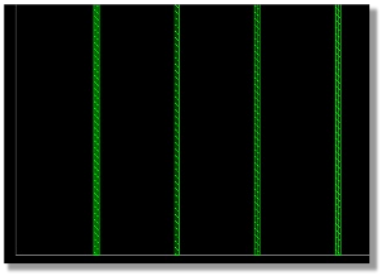
set_pg_strategy s1 -pattern {{pattern: pg_mesh_sw} {nets: {VDD}}} \
        -pg_regions [get_pg_regions switch_region*]
compile_pg -strategies s1
remove_pg_regions switch_region*
```

Commands used:
create_pg_region, create_pg_mesh_pattern, set_pg_strategy, remove_pg_regions, and compile_pg.

Copy to script editor

M4 Single Layer PG Straps With Distributed Spacing *Go to top*

This example creates single-layer metal straps on M4.



```
create_pg_mesh_pattern strap_pattern \
   -layers {{{vertical_layer: M4} {width: 0.6}\
             {pitch: 20}{spacing: interleaving} {trim: false}}}

set_pg_strategy M4_straps -core \
   -pattern {{name: strap_pattern} {nets: VDD VSS}}

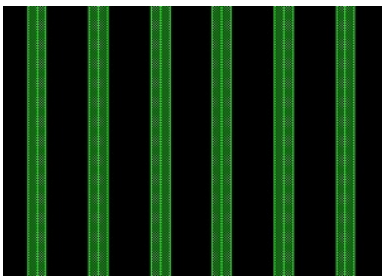
compile_pg -strategies M4_straps
```

Commands used:
create_pg_mesh_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Aligning Straps to Specific Mask Tracks. *Go to top*

This example creates single-layer metal straps on M2. The created M2 straps are aligned to mask_two tracks.



```
set_pg_mask_constraint m2c2 -track_alignment_mode aligned

create_pg_mesh_pattern  M2_wire \
 -layers {{{vertical_layer: M2}{width: 3}{spacing: interleaving} \
           {pitch: 20}{offset: 5}{trim: false} \
           {track_alignment : track}{mask: mask_two}{mask_constraint: m2c2}}}


set_pg_strategy PgStrategy_M2 -core \
       -pattern {{name: M2_wire} {nets: VDD VSS}}

compile_pg -strategies PgStrategy_M2
```

Commands used:
set_pg_mask_constraint, create_pg_mesh_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Vias Between Parallel M4 Mesh and M2 Rail *Go to top*

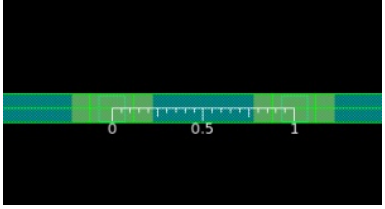
This example creates single-layer metal straps on M4 and standard cell rails on M1 and M2. Vias with dimension 1x1 and pitch 1.0 are created between parallel M4 straps and M2 rails.

```
create_pg_std_cell_conn_pattern rail_pattern -layers {M1 M2}

set_pg_strategy M1M2_rails -core \
   -pattern {{name: rail_pattern}{nets: VDD VSS}}

create_pg_mesh_pattern pg_mesh_M4 \
      -layers { \
         {{horizontal_layer: M4} {width: 0.16} {spacing: interleaving} \
          {pitch: 5.76} {offset: 0} {trim: false}} \
}

set_pg_strategy pg_mesh_strategy_M4 -core \
          -pattern {{pattern: pg_mesh_M4} {nets: {VDD VSS}} } \
          -blockage {{nets:VDD VSS}{macros:all}}
```




```
set_pg_via_master_rule staple_via -contact_code {VIA23 VIA34} \
        -allow_multiple {1.0 0} -via_array_dimension {1 1}

set_pg_strategy_via_rule pgst_via_rule  -via_rule {\
     {{{strategies: M1M2_rails} {layers: M2}} \
       {{strategies: pg_mesh_strategy_M4} {layers: M4}} \
      {via_master: staple_via} {between_parallel: true}} \
       {{intersection: undefined}{via_master: NIL}} \
}

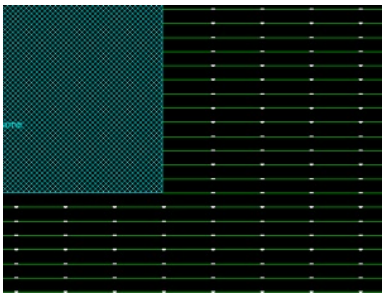
compile_pg -strategies {pg_mesh_strategy_M4 M1M2_rails} -via_rule {pgst_via_rule}
```

Commands used:
create_pg_std_cell_conn_pattern, create_pg_mesh_pattern, set_pg_strategy, set_pg_via_master_rule, set_pg_strategy_via_rule, and compile_pg.

Copy to script editor

Vertically Aligned Vias Between Parallel M4 Mesh and M2 Rail *Go to top*

This example creates single-layer metal straps on M4 and standard cell rails on M1 and M2. Vias with dimension 2x1 and pitch 10.0 are created between parallel M4 straps and M2 rails. All the created vias are vertically aligned. First via for each row is created at the X coordinate 5, which is specified by the option -snap_reference_point of the command set_pg_via_master_rule. All the remaining vias of each row are created with pitch 10.



```
create_pg_std_cell_conn_pattern rail_pattern -layers {M2 M1}

set_pg_strategy M1M2_rails -core \
   -pattern {{name: rail_pattern}{nets: VDD VSS}}

create_pg_mesh_pattern pg_mesh_M4 \
   -layers { \
             {{horizontal_layer: M4} {width: 0.16} {spacing: interleaving} \
              {pitch: 5.76} {offset: 0} {trim: false}} \
}

set_pg_strategy pg_mesh_strategy_M4 -core \
        -pattern {{pattern: pg_mesh_M4} {nets: {VDD VSS}} } \
        -blockage {{nets:VDD VSS}{macros:all}}

set_pg_via_master_rule staple_via -contact_code {VIA23 VIA34} \
        -allow_multiple {10 0} -via_array_dimension {2 1} \
        -snap_reference_point {5 0}

set_pg_strategy_via_rule rule_1  -via_rule {\
        {{{strategies: M1M2_rails} {layers: M2}} \
         {{strategies: pg_mesh_strategy_M4} {layers: M4}} \
         {via_master: staple_via} {between_parallel: true}} \
         {{intersection: undefined}{via_master: NIL}} \
}

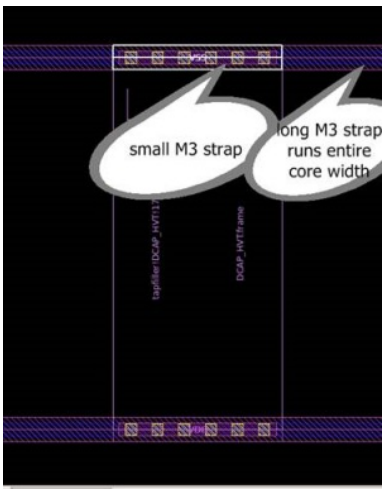
compile_pg -strategies {pg_mesh_strategy_M4 M1M2_rails} -via_rule {rule_1}
```

Commands used:
create_pg_std_cell_conn_pattern, create_pg_mesh_pattern, set_pg_strategy, set_pg_via_master_rule, set_pg_strategy_via_rule, and compile_pg.

Copy to script editor

Vias Between Parallel M3 Mesh and M1 Rail on Special Cells *Go to top*

This example creates vias between existing horizontal M3 shapes and M1 rails only on specific diode cells. Vias with dimension 6x1 are created.




```
set_app_options -name plan.pgroute.treat_fixed_stdcell_as_macro -value true

set diode_cells_R0 [get_cells "tapfiller*" -filter orientation==R0]
set diode_cells_MX [get_cells "tapfiller*" -filter orientation==MX]

set_pg_via_master_rule pgvia_6x1 -via_array_dimension {6 1} \
                   -contact_code {VIA12SQ VIA23SQ}

create_pg_mesh_pattern pg_mesh_M3 -layers { {{horizontal_layer: M3} \
                 {width: 0.11} {spacing: {1.562}}  {trim: false}} }

set_pg_strategy diodesR0 -macros $diode_cells_R0 \
                   -pattern { {name: pg_mesh_M3} {nets: {VSS VDD}} }

set_pg_strategy diodesMX -macros $diode_cells_MX \
                   -pattern { {name: pg_mesh_M3} {nets: {VDD VSS}} }

set_pg_strategy_via_rule m1m3_vias -via_rule {
            {  {{strategies: {diodesR0 diodesMX}}    } \
               {{existing: std_conn}  }  \
                  {via_master: {pgvia_6x1}} {between_parallel: true} } \
     }

compile_pg -strategies {diodesR0 diodesMX} -via_rule m1m3_vias

set_app_options -name plan.pgroute.treat_fixed_stdcell_as_macro -value false
```

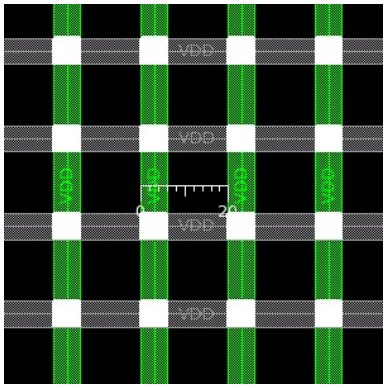
Commands used:
set_app_options, set_pg_via_master_rule, create_pg_mesh_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

PG Straps and Vias

Go to top

This example avoids the pattern-based power planning commands to manually create PG straps and insert vias. The `create_pg_strap` and `create_pg_vias` commands to manually create new PG straps on the M4 layer and insert vias between layers M4 and the preexisting M5 PG straps.




```
create_pg_strap -layer M4 -direction vertical \
   -net VDD -width 6.0 \
   -start 20 -stop 1500 -pitch 20

create_pg_vias -nets VDD \
   -within_bbox [get_attribute [get_core_area] bbox] \
   -from_layers M5 -to_layers M4
```

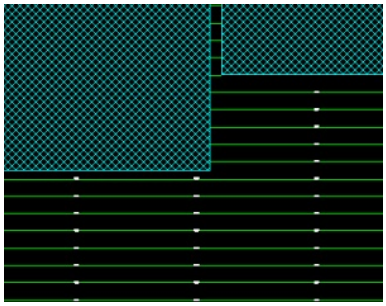
Commands used:
create_pg_strap and create_pg_vias.

Copy to script editor

Vertically Aligned Vias

Go to top

This example creates vertically aligned vias between parallel wires M4 and M2. Vias with dimension 2x1 and pitch 20 are created. All the created vias are vertically aligned. First via for each row is created at the X coordinate 10, which is specified by the option -snap_reference_point of the command set_pg_via_master_rule. All the remaining vias of each row are created with pitch 100.



```
set_pg_via_master_rule staple_via -contact_code {VIA23 VIA34} \
         -allow_multiple {20 0} -via_array_dimension {2 1} \
         -snap_reference_point {10 0}

create_pg_vias -nets {VDD VSS} -from_layers M4 -to_layers M2 -via_masters staple_via \
   -allow_parallel_objects -within_bbox [get_attr [current_block] bbox]
```

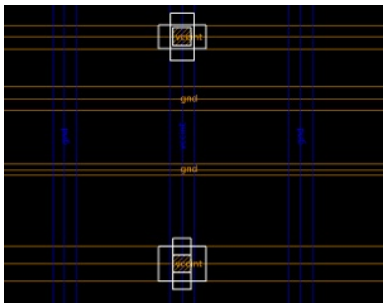
Commands used:
set_pg_via_master_rule and create_pg_vias.

Copy to script editor

Multiple ContactCodes to Repair Via DRC

Go to top

This example creates vias between M6 and M7, with contactCodes VIA67_BW114, VIA67_BW76, VIA67_BW38_UW38, and VIA67_BW21. By default, tool uses all available contactCode to fix DRC of via masters. Once a contactCode is selected to create a DRC-clean via master, the contactCode is used for all corresponding via instances, and will not be changed during DRC fixing of via instances, which can result in failing to fix DRC of via instances for some cases. To enable the tool to use multiple contactCode to fix DRC of via instances, apply the following setting: (1) apply the plan.pgroute.fix_via_drc_multiple_viadef app option, (2) define a via master rule and use the option -contact_code to specify list of available contactCode, and (3) specify the defined via master rule in the option -via_masters of create_pg_vias.




```
set_app_options -name plan.pgroute.fix_via_drc_multiple_viadef -name true

set_pg_via_master_rule vrule -contact_code \
  {VIA67_BW114 VIA67_BW76 VIA67_BW38_UW38 VIA67_BW21}

create_pg_vias -from_layers M6 -to_layers M7 -nets vccint -via_masters {vrule}
```

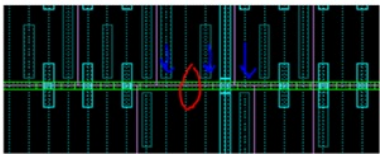
Commands used:
set_app_options, set_pg_via_master_rule and create_pg_vias.

Copy to script editor

Vias between Parallel Wires and Rails Considering Signal Routes and Standard Cells Pins

Go to top

This example creates vias between parallel wires M3 and rails M1. Vias with dimension 1x1, pitch 0054, and contactCode V1_0_8_3_49_VV and V2_3_49_25_0_VH are created. The created VDD and VSS vias have tags stv_ss and stv_vdd, respectively. All signal routes and standard cells pins are honored during DRC checking.



```
set_app_options -name plan.pgroute.honor_std_cell_drc -value true
set_app_options -name plan.pgroute.honor_signal_route_drc -value true

set_pg_via_master_rule staple_via -contact_code {V1_0_8_3_49_VV V2_3_49_25_0_VH} \
         -allow_multiple {0.054 0} -via_array_dimension {1 1}

create_pg_vias -from_layers M3 -to_layers M1 -from_types stripe \
         -to_types lib_cell_pin_connect -nets VSS \
         -allow_parallel_objects -insert_additional_vias \
         -via_masters staple_via -mark_as std_conn -tag stv_vss

create_pg_vias -from_layers M3 -to_layers M1 -from_types  stripe \
         -to_types  lib_cell_pin_connect -nets VDD \
         -allow_parallel_objects -insert_additional_vias \
```

```
-via_masters staple_via -mark_as std_conn -tag stv_vdd
```

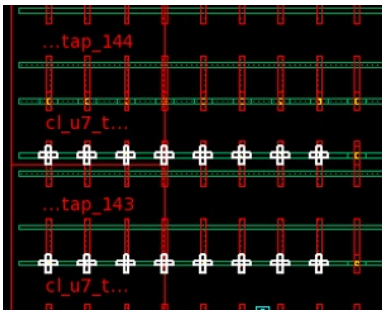
Commands used:
set_app_options, set_pg_via_master_rule and create_pg_vias.

Copy to script editor

Custom Vias at Specified Locations

Go to top

This example creates custom vias with 8 1x1 cuts at specified locations. The command create_via_def is used to define the custom vias. In the command create_pg_vias, the option -start specifies the starting points of vias for nets VDD and VSS, and option -pitch specifies pitches of the vias.




```
create_via_def via23_x8 -shapes {{v2 {-0.01 -0.01} {0.01  0.01}} {m2 {-0.04 -0.01} \
                {0.040 0.01}} {m3 {-0.011 -0.04} {0.011 0.04}} \
                                    {v2 {0.162 -0.01} {0.182 0.01}} {m2 {0.132 -0.01} \
                {0.212 0.01}} {m3 {0.161  -0.04} {0.183 0.04}} \
                                    {v2 {0.334 -0.01} {0.354 0.01}} {m2 {0.304 -0.01} \
                {0.384 0.01}} {m3 {0.333  -0.04} {0.355 0.04}} \
                                    {v2 {0.504 -0.01} {0.524 0.01}} {m2 {0.474 -0.01} \
                {0.554 0.01}} {m3 {0.503  -0.04} {0.525 0.04}} \
                                    {v2 {0.674 -0.01} {0.694 0.01}} {m2 {0.644 -0.01} \
                {0.724 0.01}} {m3 {0.673  -0.04} {0.695 0.04}} \
                                    {v2 {0.846 -0.01} {0.866 0.01}} {m2 {0.816 -0.01} \
                {0.896 0.01}} {m3 {0.845  -0.04} {0.867 0.04}} \
                                    {v2 {1.018 -0.01} {1.038 0.01}} {m2 {0.988 -0.01} \
                {1.068 0.01}} {m3 {1.017  -0.04} {1.039 0.04}} \
                                    {v2 {1.188 -0.01} {1.208 0.01}} {m2 {1.158 -0.01} \
                {1.238 0.01}} {m3 {1.187  -0.04} {1.209 0.04}} \
}

create_pg_vias -from_layers m2 -to_layers m3 -via_masters via23_x8 -nets {VDDL VSSL} \
-within_bbox [get_attribute [get_core_area] bbox] -start {{0.17 0.28} {0.17 0.04}} \
-pitch {{1.3 68 0.72} {1.368 0.72}}
```

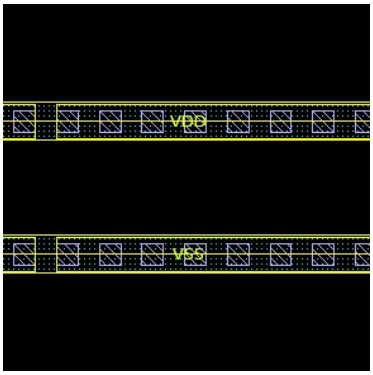
Commands used:
create_via_def, and create_pg_vias.

Copy to script editor

Stapling Vias

Go to top

This example inserts stapling vias with dimension 10x1 between the adjacent layers M1 and M2 in areas where the layers overlap.




```
create_pg_std_cell_conn_pattern std_cell_rail -layers {M1 M2}

set_pg_strategy std_cell_rail -pattern {\
   {pattern: std_cell_rail} {nets: VDD VSS}} \
   -core

compile_pg -strategies std_cell_rail

set core_bbox [get_attribute [get_core_area] bbox]

create_pg_stapling_vias -regions $core_bbox -nets {VDD VSS} \
  -from_layer M2 -to_layer M1 \
  -from_shapes [get_shapes -filter "layer_name == M2"] \
  -to_shapes [get_shapes -filter "layer_name == M1"] \
  -max_array_size 10
```

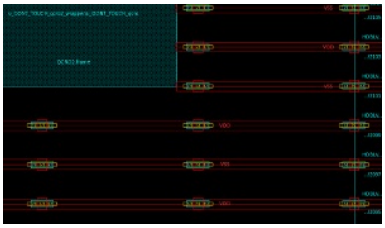
Commands used:
create_pg_std_cell_conn_pattern, set_pg_strategy, compile_pg, and create_pg_stapling_vias.

Copy to script editor

Stapling Vias Between Non-Adjacent Layers

Go to top

This example inserts stapling vias between non-adjacent layers M3 and M1 in areas where the layers overlap. Via master rules via1_staple and via2_staple are defined to vias between M2 and M1, and M3 and M2, respectively. Same pitch values are used (defined in -allow_multiple) for the via master rules via1_staple and via2_staple to ensure via1 and via2 are stacked. The option -snap_reference_point is used to align created stapling vias.



```
set m3_shapes [get_shapes -filter "layer_name == M3"]
set m1_shapes [get_shapes -filter "layer_name == M1"]

set_pg_via_master_rule via1_staple -contact_code via1 \
  -via_array_dimension {2 1} -allow_multiple {0.5 0} \
  -snap_reference_point {0 0}

set_pg_via_master_rule via2_staple \
  -contact_code via2 -via_array_dimension {1 1} \
  -allow_multiple {0.5 0} -snap_reference_point {0 0}

create_pg_stapling_vias -nets {VDD VSS} \
  -from_layer M3 -to_layer M1 \
  -from_shapes $m3_shapes -to_shapes $m1_shapes \
  -via_masters {via1_staple via2_staple}
```

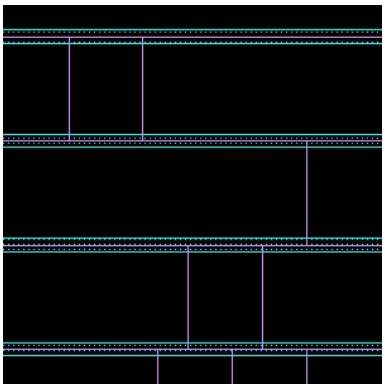
Commands used:
set_pg_via_master_rule, and create_pg_stapling_vias.

Copy to script editor

PG Standard Cell Rails

Go to top

This example inserts standard cells rails for nets VDD and VSS on layer M1. The width is not specified, so the tool creates the rails with the same width as the standard cell pins.



```
create_pg_std_cell_conn_pattern rail_pattern -layers M1

set_pg_strategy M1_rails -core \
   -pattern {{name: rail_pattern}{nets: VDD VSS}}

compile_pg -strategies M1_rails
```

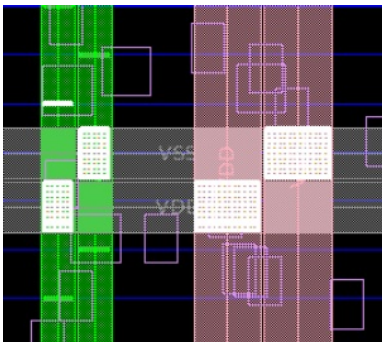
Commands used:
create_pg_std_cell_conn_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

PG Mesh and Standard Cell Rails

Go to top

This example creates a mesh on vertical layers M4 and M6 and horizontal layer M5, and standard cells rails for nets VDD and VSS on layer M1. Vias with dimension 6x1 are created between mesh on layer M4 and M1 rails. Default vias are created between layers in the mesh.



```
create_pg_mesh_pattern mesh_pattern \
   -layers {{{vertical_layer: M6} {width: 4} {pitch: 40} {offset: 20}} \
            {{vertical_layer: M4} {width: 2} {pitch: 35} {offset: 20}} \
            {{horizontal_layer: M5} {width:  3} {pitch: 40} {offset: 15}}}

set_pg_strategy M6M5M4_mesh -core \
   -pattern {{name: mesh_pattern} {nets: VDD VSS}}

create_pg_std_cell_conn_pattern rail_pattern -layers M1

set_pg_strategy M1_rails -core \
   -pattern {{name: rail_pattern}{nets: VDD VSS}}

set_pg_via_master_rule VIA_6x1 -via_array_dimension {6 1}

set_pg_strategy_via_rule via_rule  -via_rule {\
    {{{strategies: M6M5M4_mesh} {layers: M4}} \
     {{strategies: M1_rails} {layers: M1}} \
     {via_master: VIA_6x1}} \
       {{intersection: undefined}{via_master: NIL}} \
}

compile_pg -strategies {M6M5M4_mesh M1_rails} -via_rule {via_rule}
```

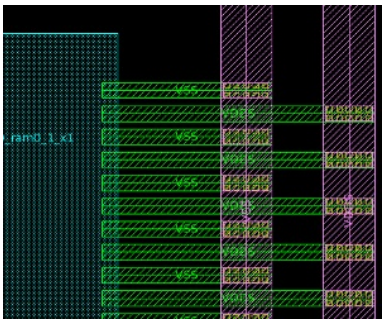
Commands used:
create_pg_mesh_pattern, create_pg_std_cell_conn_pattern, set_pg_via_master_rule, set_pg_strategy_via_rule, set_pg_strategy, and compile_pg.

Copy to script editor

PG Macro and Pad Pin Connection

Macro Pin Connection Go to top

This example creates PG pin connections for hard macros. M5 and M6 wires are used for horizontal and vertical connection. The width of horizontal and vertical wires are 0.3 microns. Only scattered pin are connected.



```
set macros [get_cells \
   -physical_context -filter "is_hard_macro  && !is_physical_only"]
create_pg_macro_conn_pattern macro_connect_pattern \
   -pin_conn_type scattered_pin -nets {VDD VSS} \
   -width {0.3 0.3} -layers {M5 M6}
set_pg_strategy macro_connect \
   -pattern {{name: macro_connect_pattern}{nets: VDDS VSS}} \
   -macros "$macros"
compile_pg -strategies macro_connect
```

Commands used:
create_pg_macro_conn_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Pad Pin Connection Go to top

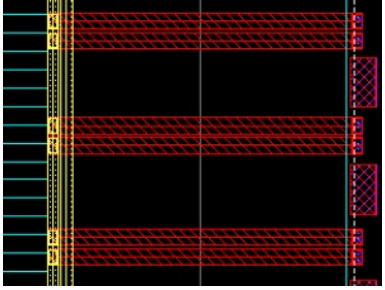
This example creates PG pin connections for I/O pad IO_E1VCC_0. M3 and M4 wires are used for horizontal and vertical connection. Only scattered M2 pin are connected. The plan.pgroute.treat_pad_as_macro app option is used to also connect pins not overlapping with pad boundaries. Without the app option, only pins overlapping with pad boundaries are connected. The strategy via rule macro_conn_via_rule is specified and used in compile_pg such that vias are created only at the intersections between connecting wires and M4 existing wires.

```
create_pg_macro_conn_pattern hm_pattern -pin_conn_type scattered_pin \
               -layers {M3 M4} -nets {VSS} -pin_layers {M2}

set_app_options -name plan.pgroute.treat_pad_as_macro -value true

set_pg_strategy macro_conn -macros [get_cells IO_E1VCC_0]
               -pattern {{name: hm_pattern} {nets: {VDD VSS}}}

set_pg_strategy_via_rule macro_conn_via_rule \
```



```
            -via_rule { \
               {{strategies: macro_conn}}{{existing: all} {layers: M4}} \
                 {via_master: default}} \
                {{intersection: undefined}{via_master: NIL}} \
          }

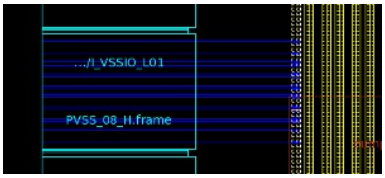
compile_pg -strategies macro_conn -via_rule macro_conn_via_rule -tag test
```

Commands used:
create_pg_macro_conn_pattern, set_pg_strategy_via_rule, set_pg_strategy, and compile_pg.

Copy to script editor

Pad Pin Connection to Rings *Go to top*

This example creates PG pin connections to the PG ring for I/O pad I_VSSIO_L01. The vias between created shapes and macro pins as well as target wires are controlled by the via rule defined in create_pg_macro_conn_pattern command. In this example, the via rule setting -via_rule {{intersection: all} {via_master: NIL}} is to ensure there is no vias created on pad pins. The plan.pgroute.hmpin_connection_target_layers application option is used to find the closest D6 shapes as target wires for connection. The strategy via rule rule1 is to define vias between created shapes and existing wires. In this example, it defined vias between created shapes on layer G1 and existing rings on layer D6.




```
set_app_options -name plan.pgroute.hmpin_connection_target_layers -value D6

create_pg_macro_conn_pattern io_to_ring -pin_conn_type scattered_pin \
    -pin_layers {G1} -layers {G1 D6} -width 0.7 \
    -via_rule {{{intersection: all} {via_master: NIL}}}

set_pg_strategy s_io_to_ring -macros I_VSSIO_L01 \
    -pattern {{name: io_to_ring}{nets: VSSIO}}

set_pg_strategy_via_rule rule1 -via_rule { \
   {{{strategies: s_io_to_ring}{layers: G1}} {{existing: ring}{layers: D6}} \
    {via_master: default}} {{intersection: undefined} {via_master: NIL}}}

compile_pg -strategies s_io_to_ring -via_rule rule1
```

Commands used:
set_app_options, create_pg_macro_conn_pattern, set_pg_strategy_via_rule, set_pg_strategy, and compile_pg.

Copy to script editor

Tap Cell Pin Connection *Go to top*

This example creates PG pin connections for tap cells. In the design, existing M3 vertical wires intersect with M2 PG pins of the tap cell WELLTAP_3_R3_C0_8138. However, there are no vias between the wires and pins. The plan.pgroute.treat_cell_type_as_macro app option is used to have PG router to make PG connection for tap cells. In the macro connection pattern, connections are specified between M2 pins and M2&M3 wires with contact code iccII_M3_M2_0_050_x_0_054_WELLTAP. Since existing wires overlap with the pins, tool only create s vias to connect the pins. No wires will be created for the connection.




```
set cellPtr WELLTAP_3_R3_C0_8138

set_app_options -name plan.pgroute.treat_cell_type_as_macro -value "well_tap"

create_pg_macro_conn_pattern P_tapcell \
       -pin_conn_type scattered_pin \
       -nets          "gnds vdds" \
       -pin_layers    M2 \
       -layers        "M2 M3" \
       -via_rule {{intersection: adjacent} {via_master: iccII_M3_M2_0_050_x_0_054_WELLTAP}}

set_pg_strategy S_tapcell \
       -macros [get_cells $cellPtr] \
       -pattern   "{name: P_tapcell}{nets: gnds vdds}"

compile_pg -strategies S_tapcell
```

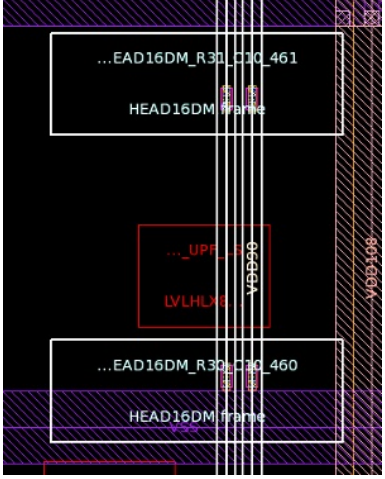
Commands used:
set_app_options, create_pg_macro_conn_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Special Patterns

Alignment Straps for Switch Cells *Go to top*

This example creates vertical PG alignment straps to connect VDD pins on layer M2 of switch cells with master HEADBUFTIE42Q_D3. The vertical alignment straps are created on D2 layer with width 0.51 microns. Two alignment straps with pitch 0.61 microns are created for each column of switch cells. Default vias are used to connect from D2 alignment straps to M2 pins of switch cells.




```
create_pg_special_pattern SW_pat -insert_power_switch_alignment_straps { \
      {lib_cells : HEADBUFTIE42Q_D3} {layer : D2} {width : 0.51} \
      {direction : vertical} {pin_layers : M2} {number : 2} \
      {pitch : 0.61} {offset : 1.5} }

set_pg_strategy SW_strategy -core -pattern { {name: SW_pat} {nets: VDD} }

set_pg_strategy_via_rule D2_SW_MESH_VIA_RULE \
      -via_rule { {{intersection: all}{via_master: default}} }

compile_pg -strategies SW_strategy -via_rule D2_SW_MESH_VIA_RULE -tag SW_MESH
```

Commands used:
create_pg_special_pattern, set_pg_strategy, set_pg_strategy_via_rule, and compile_pg.

Copy to script editor

Channel Straps *Go to top*

This example creates straps in channels between two macros, two placement blockages, and one macro and one placement blockage. The channel straps are created on layer M1Z with width 2 microns, and spacing 2 microns. The straps are created to only the channels whose widths are larger than 15 microns.




```
create_pg_special_pattern channelPattern -insert_channel_straps { \
   {layer: M1Z}{direction: vertical}{width: 2}{spacing: 2}{channel_threshold: 15} \
   {check_one_layer: true} {channel_between_objects: {macro placement_blockage}}}

set_pg_strategy s_channel -voltage_areas PD_VDD11_I \
                -pattern {{pattern: channelPattern}{nets:VSS VDD11_I}}

compile_pg -strategies s_channel -tag channel
```

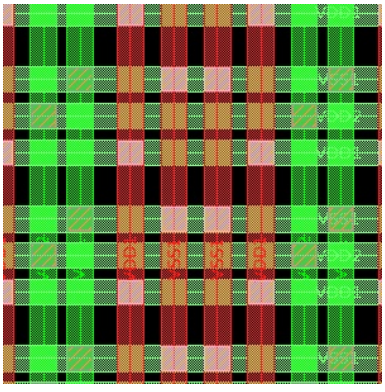
Commands used:
create_pg_special_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

PG Composite Patterns

Composite Pattern from Three Wire Patterns *Go to top*

This example creates a composite pattern that contains three individual pattern descriptions. The `create_pg_wire_pattern` command defines the base pattern, and the `create_pg_composite_pattern` command defines the parameters for three pattern sets. Note that create_pg_composite_pattern is using symbolic net names rather than real net names. In the example below, The net names in create_pg_composite_pattern POWER1, POWER2, and POWER3 are symbolic net names, while the net names in set_pg_strategy VDD1 VSS1 VSS1 are real net names. There is one-to-one mapping between the real net names in set_pg_strategy and the symbolic net names in create_pg_composite_pattern



```
create_pg_wire_pattern base_wire_pattern -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}

create_pg_composite_pattern composite1 -nets {POWER1 POWER2 GROUND1} \
   -add_patterns { \
 {{name: base_wire_pattern} {nets: POWER1 GROUND1 GROUND1 POWER1} \
  {parameters: {M6 vertical 1.5 15 1}}{offset: 0}} \
 {{name: base_wire_pattern} {nets: POWER2 GROUND1} \
  {parameters: {M4 vertical 1.5 15 0.6}}{offset: 10}} \
 {{name: base_wire_pattern} {nets: POWER1 POWER2 GROUND1} \
  {parameters: {M5 horizontal 1.5 8 0.6}}{offset: 2.5}} \
}

set_pg_strategy core_strategy \
    -pattern {{name: composite1} \
              {nets: VDD1 VSS1 VSS1} {offset: {5 5}}} -core

compile_pg -strategies core_strategy
```

Commands used:
create_pg_wire_pattern, create_pg_composite_pattern, and set_pg_strategy, and compile_pg.

Copy to script editor

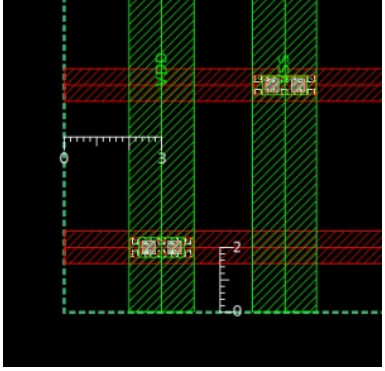
Composite Pattern with PG Region and Offset Defined in Wire Pattern *Go to top*

This example creates a composite pattern that contains three individual pattern descriptions. The `create_pg_wire_pattern` command defines the base pattern, and the `create_pg_composite_pattern` command defines the parameters for three pattern sets. The offset 3, 3, and 2 are applied to the first, second, and third pattern description. Two PG regions are defined and used for strategy mesh_strategy as routing areas.

```
create_pg_wire_pattern stra_base -layer @l -direction @d -width @w \
   -spacing @s -pitch @p -trim false \
   -parameters {l d w p s}

create_pg_composite_pattern pd_gsm_mesh -nets {vdd gnd} \
  -add_patterns {\
     {{pattern:stra_base}{nets:{vdd gnd} } \
       {parameters: {M4 vertical 2 10 1.8}} {offset: 3}} \
     {{pattern:stra_base}{nets:{vdd gnd} } \
       {parameters: {M6 vertical 2 10.2 1.8}} {offset: 3}} \
     {{pattern:stra_base}{nets:vdd gnd } \
       {parameters: {M5 horizontal 1 10 interleaving}}{offset:2}} \
}

create_pg_region region1 \
```



```
    -polygon {{599.505 1140.405} {933.175 1140.405} {933.175 838.345} {599.505 838.345} }
create_pg_region region2 \
    -polygon {{1147.430 1172.015} {1649.690 1172.015} {1649.690 725.950} {1147.430 725.950} }

set_pg_strategy mesh_strategy -pg_regions {region1 region2} \
             -pattern {{pattern: pd_gsm_mesh}{nets: {VDD VSS}}}

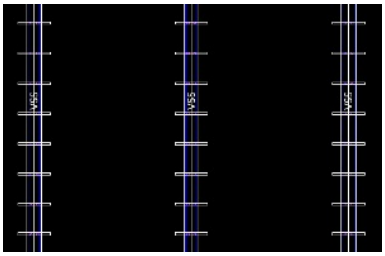
compile_pg -strategies mesh_strategy
```

Commands used:
create_pg_wire_pattern, create_pg_composite_pattern, and create_pg_region, and set_pg_strategy, and compile_pg.

Copy to script editor

Composite Pattern with Custom Vias Between Parallel Wires *Go to top*

This example creates a composite pattern that contains two wire patterns on layers M7 and M9. Two custom viaDef PgVia_via7_CUST and PgVia_via8_CUST are created. Vias with the custom ViaDef are added between parallel wires M7 and M9.



```
create_via_def PgVia_via7_CUST -cut_layer VIA7 -cut_size {0.038 0.038} \
            -upper_enclosure {0.18 0.001} -lower_enclosure {0.18 0.001} \
            -min_rows 1 -min_columns 2 -min_cut_spacing 0.074
create_via_def PgVia_via8_CUST -cut_layer VIA8 -cut_size {0.038 0.038} \
            -upper_enclosure {0.18 0.001} -lower_enclosure {0.18 0.001} \
            -min_rows 1 -min_columns 2 -min_cut_spacing 0.074
set_pg_via_master_rule PgVia_via78_rule_VSS -contact_code {PgVia_via7_CUST PgVia_via8_CUST} \
            -via_array_dimension {2 1} -allow_multiple {2.508 0.48} -cut_spacing {0.064 0.090} \
            -snap_reference_point {2.736 0.48}

create_pg_wire_pattern PgPat_wire_strap -layer @l -direction @d -width @w -pitch @p -spacing @s \
            -parameters {l d w p s}
create_pg_composite_pattern pattern_VSS -nets VSS -add_patterns {\
{{pattern: PgPat_wire_strap} {nets: VSS} {parameters: {M7 vertical {00.190 } 02.508 interleaving}} {offset: 02.508}} \
{{pattern: PgPat_wire_strap} {nets: VSS} {parameters: {M9 vertical {00.230 } 02.508 interleaving}} {offset: 02.508}} \
} \
-via_rule { \
{ {pattern_id: 1} {pattern_id: 2}  {via_master: PgVia_via78_rule_VSS} {between_parallel: true}} \
{{intersection: undefined} {via_master: NIL}} \
}

set_pg_strategy strategy_VSS -core -pattern {{name: pattern_VSS} {nets: VSS } {offset: {0 0}}} \
                             -extension { {{stop: design_boundary_and_generate_pin}{direction: L B R T}} }
set_pg_strategy_via_rule blockMacroViaVSS -via_rule {{{{strategies: strategy_VSS} {nets: VSS}} {via_master:NIL}}}
compile_pg -strategies strategy_VSS -via_rule blockMacroViaVSS
```

Commands used:
create_via_def, set_pg_via_master_rule, create_pg_wire_pattern, create_pg_composite_pattern, set_pg_strategy_via_rule, set_pg_strategy, and compile_pg.

Copy to script editor

Composite Pattern with Custom Via *Go to top*

This example defines custom via defs and creates the corresponding custom vias in the middle of two vertical wires. Two custom via def m2_m3 and m3_m4 are created. And two via master rules offset_m2_m3 and offset_m3_m4 are defined to shift the created vias by 0.08 microns horizontally. In composite pattern core_pattern, custom vias are created between net id 1 and 3 (the first VDDS and VSS) of the first instantiated wire pattern and net id 1 and 2 (VDDS and VSS) of the second instantiated pattern. No vias are created for all other intersections. After that, vias with via def m2_m3 and m3_m4 can be created in the middle of two vertical wires for nets VVDD and VSS.

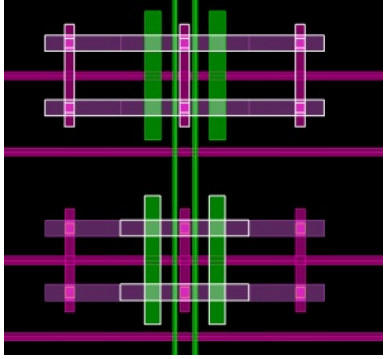
```
create_via_def m2_m3 \
  -shapes { \
   {M3 {-1.392 -.4} {1.392 -.24}} \
   {M3 {-1.392 .24} {1.392 .4}} \
   {M2 {-1.20 -.512} {-1.104 .512}} \
   {M2 {-.048 -.512} {.048 .512}} \
   {M2 {1.104 -.512} {1.2 .512}} \
   {VIA2 {-1.2 -.368} {-1.104 -.272}} \
   {VIA2 {-.048 -.368} {.048 -.272}} \
   {VIA2 {1.104 -.368} {1.2 -.272}} \
   {VIA2 {-1.2 .272} {-1.104 .368}} \
   {VIA2 {-.048 .272} {.048 .368}} \
   {VIA2 {1.104 .272} {1.2 .368}} \
   }

create_via_def m3_m4 \
  -shapes { \
   {M3 {-.640 -.4} {.64 -.24}} \
   {M3 {-.64 .24} {.64 .4}} \
   {M4 {-.4 -.64} {-.24 .64}} \
   {M4 {.24 -.64} {.4 .64}} \
   {VIA3 {-.4 .24} {-.24 .4}} \
   {VIA3 {.24 .24} {.4 .4}} \
   {VIA3 {-.4 -.4} {-.24 -.24}} \
   {VIA3 {.24 -.4} {.4 -.24}}\
   }

set_pg_via_master_rule offset_m2_m3 \
  -contact_code m2_m3 \
  -offset {0.1 0.0}

set_pg_via_master_rule offset_m3_m4 \
  -contact_code m3_m4 \
  -offset {0.1 0.0}

create_pg_wire_pattern stra_base \
```



```
  -layer @l \
  -direction @d \
  -width @w \
  -spacing @s \
  -pitch @p \
  -parameters {l d w s p}

create_pg_composite_pattern core_pattern \
  -nets {VSS VDDS} \
  -add_patterns { \
    {{pattern: stra_base} {nets: VDDS VDDS VSS VSS} \
     {parameters: {M4 vertical 0.040 {0.120 6.040 0.120} 47.60}} {offset: 10.856}} \
    {{pattern: stra_base} {nets: VDDS VSS} \
     {parameters: {M2 horizontal 0.084 0.996 1.840 }} {offset: 0.42}} \
  } \
  -via_rule { \
    {{{pattern_id: 1} {net_id: {1 3}}} {{pattern_id: 2} {net_id: {1 2}}} \
      {via_master: offset_m2_m3 offset_m3_m4}} \
    {{intersection: undefined} {via_master: NIL}} \
  }

set_pg_strategy core_strategy \
    -pattern {{name: core_pattern} \
              {nets: VSS VDDS}} -core

compile_pg -strategies core_strategy -ignore_drc
```

Commands used:
create_via_def, set_pg_via_master_rule, create_pg_wire_pattern, create_pg_composite_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Composite Pattern with Tcl variables *Go to top*

This example creates a composite pattern that contains four individual pattern descriptions. The `create_pg_wire_pattern` command defines the base pattern, and the `create_pg_composite_pattern` command defines the parameters for four pattern sets. Tcl variables m2_order, m3_order, m4_order, and m5_order are used to define the used nets for layers M2, M3, M4, and M5, respectively. The Tcl variable compositeMeshRuleSpec is used to specify the contents of the composite pattern core_pattern.




```
set m2_order "VDD VSS"
set m3_order "VSS VDD"
set m4_order "VSS VSS"
set m5_order "VDD VDD"

create_pg_wire_pattern wire_patt -spacing interleaving -layer @l -direction @d -width @w \
                                 -pitch @p -parameters {l d w p}

set compositeMeshRuleSpec "{{pattern: wire_patt}{[list nets: $m2_order]} \
                            {parameters: {M2 vertical 1 10}}{offset:  @m2o}} \
                           {{pattern: wire_patt}{[list nets: $m3_order]} \
                            {parameters: {M3 horizontal 1.5 12}}{offset:  @m3o}} \
                           {{pattern: wire_patt}{[list nets: $m4_order]} \
                            {parameters: {M4 vertical 2 15}}{offset:  @m4o}} \
                           {{pattern: wire_patt}{[list nets: $m5_order]} \
                            {parameters: {M5 horizontal 2 15}}{offset:  @m5o}}"

create_pg_composite_pattern core_pattern -nets {VDD VSS} \
                             -add_patterns $compositeMeshRuleSpec \
                             -parameters {m2o m3o m4o m5o}

set_pg_strategy core_strategy \
    -pattern {{name: core_pattern} \
              {nets: VDD VSS} {parameters: {5 5 5 5}}} -core

compile_pg -strategies core_strategy
```

Commands used:
create_pg_wire_pattern, create_pg_composite_pattern, and set_pg_strategy, and compile_pg.

Copy to script editor

Composite Pattern for Advanced Nodes *Go to top*

This example creates a power grid that connects to M0 rails for an advanced technology node. The M0 rails are created with the create_pg_std_cell_conn_pattern command in the example below. Since the M0 rails are high resistance, it is necessary to connect to the rails at frequent intervals. This is done by creating vertical M1 straps that are close together. The M1 straps are the same width as the regular routing, and they are on track. That way, they don't take unnecessary amounts of routing resource. The twire_base wire_pattern in the example declares on-track power straps.

The example uses M3 power straps that are parallel to the M1 straps. The M3 straps are also on track and nearly overlap the M1 straps. These are needed to add more current carrying capacity. Small segments of M2 are placed between the M1 and M3 in order to connect them. The seg_base wire pattern declares the M2. The M2 segments are on track and minimum width so they don't block routing. In the picture, M0 is yellow, M1 is blue, M2 is orange, and M3 is red.

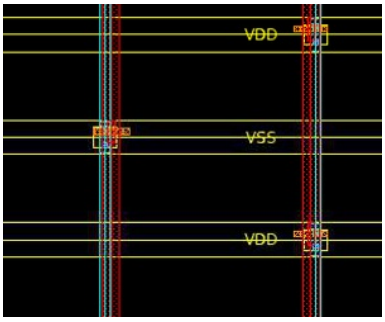
A wider M4/M5 grid above the M3 carries more current. You will probably also want more straps on the higher metals. The higher metal is not shown in this example.

All M1 through M5 metal is declared in the comp_pat composite pattern, which defines the alignment between the layers and how to place the vias. The challenging task in this example is to define the small pieces of M2. The M2 VDD and VSS parts are declared separately and aligned so that the connection between M1 and M3 occurs directly over the point where M1 connects to the M0 rails. It takes a bit of iteration to get the parameter values just right. The via rules in comp_pat are constructed so that there is no direct via staple between non-adjacent layers. Adding non-adjacent staples would severely affect routability.

The via_rule_strategy vr1 is used to connect the M1 from comp_pat to the M0 rails. Again, the rule avoids direct via staples from non-adjacent layers.

```
# create the power grid
create_pg_std_cell_conn_pattern std_cell_pattern
set_pg_strategy rails -core -pattern {{pattern: std_cell_pattern}{nets: {VDD VSS}}}

create_pg_wire_pattern wire_base -layer @l -direction @d -width @w \
          -pitch @p -spacing @s -parameters {l d w p s}
```




```
create_pg_wire_pattern twire_base -layer @l -direction @d -width @w \
          -pitch @p -spacing @s -track_alignment @t -parameters {l d w p s t}
create_pg_wire_pattern seg_base -layer @l -direction @d -width @w -pitch @p \
        -low_end_reference_point @from -high_end_reference_point @to \
        -track_alignment track -parameters {l d w p from to}

create_pg_composite_pattern comp_pat -nets {VDD VSS} \
  -add_patterns { \
    {{pattern: wire_base} {nets: VDD VSS} {parameters: {M5 vertical .3 5 interleaving}}} \
    {{pattern: wire_base} {nets: VDD VSS} {parameters: {M4 horizontal .3 5 interleaving}}} \
    {{pattern: twire_base} {nets: VDD VSS} {parameters: {M3 vertical 0.060 1.4 interleaving auto}}} \
    {{pattern: seg_base} {nets: VDD} {parameters: {M2 horizontal 0.020 {1.4 0.720} -0.060 0.060}} {offset 0 0}} \
    {{pattern: seg_base} {nets: VSS} {parameters: {M2 horizontal 0.020 {1.4 0.720} -0.060 0.060}} {offset 0.7 0.360}} \
    {{pattern: twire_base} {nets: VDD VSS} {parameters: {M1 vertical 0.038 1.4 interleaving track}}} \
  } \
  -via_rule { \
    {{{pattern_id: 1}} {{pattern_id: 2}} {via_master: default}} \
    {{{pattern_id: 2}} {{pattern_id: 3}} {via_master: default}} \
    {{{pattern_id: 3}} {{pattern_id: 4}} {via_master: default}} \
    {{{pattern_id: 3}} {{pattern_id: 5}} {via_master: default}} \
    {{{pattern_id: 4}} {{pattern_id: 6}} {via_master: default}} \
    {{{pattern_id: 5}} {{pattern_id: 6}} {via_master: default}} \
    {{intersection: undefined} {via_master: NIL}} \
  }

set_pg_strategy s1 -pattern {{pattern: comp_pat} {nets: {VDD VSS}}} \
         -core -blockage {{{macros: all}}}

set_pg_strategy_via_rule vr1 -via_rule { \
    {{{strategies: rails}{layers: M0}}{{strategies: s1}{layers: M1}} {via_master: default}} \
    {{intersection: undefined}{via_master: NIL}} \
  }

compile_pg -ignore_drc -strategies {rails s1} -via_rule vr1
#done with power grid
```

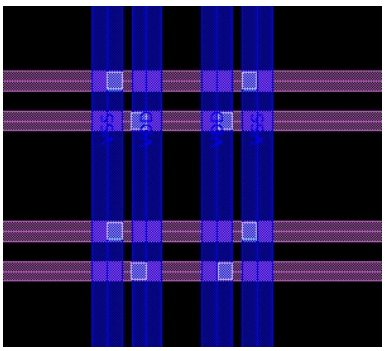
Commands used:
create_pg_wire_pattern, create_pg_std_cell_conn_pattern, create_pg_composite_pattern, set_pg_strategy, set_pg_strategy_via_rule, and compile_pg.

Copy to script editor

Center-Aligned Vias *Go to top*

This example creates horizontal power straps on layer M3 and vertical power straps on layer M8. Stacked vias connect the horizontal and vertical straps, and the vias are pushed toward the center of each VDD/VSS vertical power strap pair to save routing resources beneath the strap. The `create_pg_wire_pattern` command defines the base pattern, and the `create_pg_composite_pattern` command defines the parameters for the horizontal and vertical PG routes.

Full Example



```
set_pg_via_master_rule via_5x6_shift_right \
   -via_array_dimension {5 6} -offset {0.8 0}

set_pg_via_master_rule via_5x6_shift_left \
   -via_array_dimension {5 6} -offset {-0.8 0}

create_pg_wire_pattern wire_base -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}

create_pg_composite_pattern center_aligned_vias -nets {VDD VSS} \
   -add_patterns { \
      {{pattern: wire_base} {nets: VSS VDD VDD VSS} \
       {parameters: {M8 vertical 3 30 {1 4 1}}}{offset: 4}} \
      {{pattern: wire_base} {nets: VDD VSS} \
       {parameters: {M3 horizontal 2 15 2}}{offset: 4}} \
   } \
   -via_rule { \
      {{{pattern_id: 1}{net_id: 1}} \
       {{pattern_id: 2}{net_id: 2}} \
       {via_master: via_5x6_shift_right}} \
      {{{pattern_id: 1}{net_id: 2}} \
       {{pattern_id: 2}{net_id: 1}} \
       {via_master: via_5x6_shift_left}} \
      {{{pattern_id: 1}{net_id: 3}} \
       {{pattern_id: 2}{net_id: 1}} \
       {via_master: via_5x6_shift_right}} \
      {{{pattern_id: 1}{net_id: 4}} \
       {{pattern_id: 2}{net_id: 2}} \
       {via_master: via_5x6_shift_left}} \
      {{intersection: undefined}{via_master: NIL}} \
  }

set_pg_strategy core_strategy \
    -pattern {{name: center_aligned_vias} \
              {nets: VDD VSS } {offset: {5 5}}} -core

compile_pg -strategies core_strategy
```

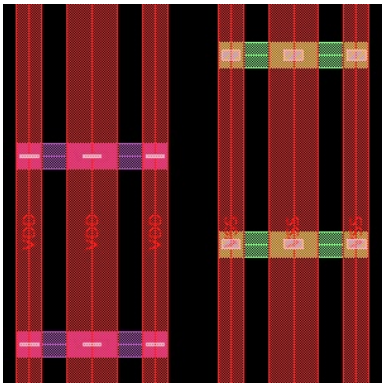
Commands used:
set_pg_via_master_rule, create_pg_wire_pattern, create_pg_composite_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Bridging Straps *Go to top*

This example creates bridging straps on layers M3 and M5 to connect vertical power straps on layer M6. The bridging straps are created for both VDD and VSS.

Full Example



```
set_pg_via_master_rule via63_5x1 \
   -contact_code {VIA34 VIA45 VIA56} \
   -via_array_dimension {5 1}

set_pg_via_master_rule via65_5x3 \
   -contact_code {VIA56} \
   -via_array_dimension {5 3}

create_pg_wire_pattern wire_base -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}

create_pg_wire_pattern segment_base -layer @l -direction @d \
   -width @w -low_end_reference_point 0 \
   -high_end_reference_point @len -pitch @p \
   -parameters {l d w len p}

create_pg_composite_pattern bridging_straps -nets {VDD VSS} \
   -add_patterns { \
   {{pattern: wire_base} {nets: VDD VSS} \
    {parameters: {M7 horizontal 2 60 2}}{offset: 13}} \
   {{pattern: wire_base} {nets: VDD VDD VDD VSS VSS VSS} \
    {parameters: {M6 vertical {2 4 2 2 4 2} 45 {2 2 4 2 2} }}{offset: 4}} \
   {{pattern: segment_base} {nets: VDD} \
    {parameters: {M3 horizontal 2 12 {45 15}}} {offset: {3 2}}} \
   {{pattern: segment_base} {nets: VSS} \
    {parameters: {M5 horizontal 2 12 {45 15}}} {offset: {19 10}}} } \
   -via_rule { \
       {{layers: M7}{layers: M6} {via_master: default}} \
       {{layers: M6}{layers: M5} {via_master: via65_5x3}} \
       {{layers: M6}{layers: M3} {via_master: via63_5x1}} \
       {{intersection: undefined} {via_master: NIL}} \
  }

set_pg_strategy core_strategy \
    -pattern {{name: bridging_straps} \
                 {nets: VDD VSS } {offset: {5 5}}} -core

compile_pg -strategies core_strategy
```

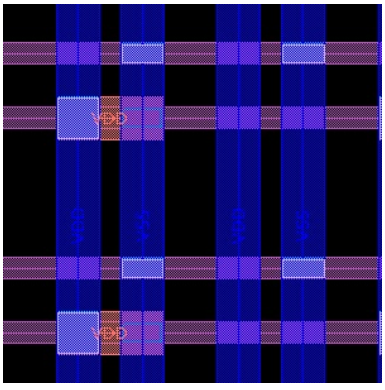
Commands used:
set_pg_via_master_rule, create_pg_wire_pattern, create_pg_composite_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Via Bridging *Go to top*

This example creates a complex via bridging structure to connect the vertical VDD strap on layer M8 with the horizontal VDD strap on layer M3. A 13x13 via array connects the vertical VDD strap on layer M8 with the short horizontal metal segment created on layer M7. A smaller 6x12 stacked via array connects the short M7 strap with the horizontal VDD strap on layer M3.

Full Example



```
set_pg_via_master_rule via73_12x6_shift_right \
   -contact_code {VIA34 VIA45 VIA56 VIA67} \
   -via_array_dimension {12 6} -offset {3 0}

create_pg_wire_pattern wire_base -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}

create_pg_wire_pattern segment_base -layer @l -direction @d \
   -width @w -low_end_reference_point 0 \
   -high_end_reference_point @len -pitch @p \
   -parameters {l d w len p}

create_pg_composite_pattern via_bridging -nets {VDD VSS} \
   -add_patterns { \
    {{pattern: wire_base} {nets: VDD VSS} \
     {parameters: {M8 vertical 4 15 2}}{offset: 4}} \
    {{pattern: wire_base} {nets: VDD VSS} \
     {parameters: {M3 horizontal 2 20 4}}{offset: 4}} \
    {{pattern: segment_base} {nets: VDD} \
     {parameters: {M7 horizontal 4 10 {30 20}}} {offset: {2 4}}} \
    } \
   -via_rule { \
    {{{pattern_id: 1}{net_id: 1}}
     {{pattern_id: 2}{net_id: 1}} {via_master: NIL}} \
    {{{pattern_id: 1}{net_id: 2}}
     {{pattern_id: 2}{net_id: 2}} {via_master: default}} \
    {{{pattern_id: 1}{net_id: 1}}
     {{pattern_id: 3}{net_id: 1}} {via_master: default}} \
    {{{pattern_id: 2}{net_id: 1}}
     {{pattern_id: 3}{net_id: 1}}
     {via_master: via73_12x6_shift_right} {between_parallel: true}} \
     {{intersection: undefined} {via_master: NIL}} \
 }

set_pg_strategy core_strategy \
    -pattern {{name: via_bridging} \
              {nets: VDD VSS } {offset: {5 5}}} -core

compile_pg -strategies core_strategy
```

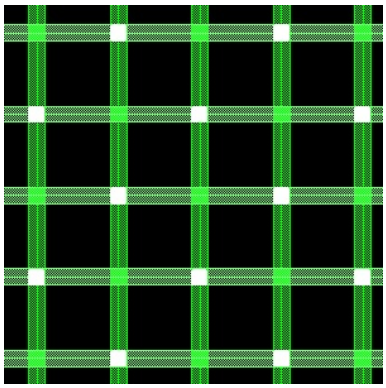
Commands used:
create_pg_wire_pattern, create_pg_composite_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Checkerboard Via Pattern *Go to top*

This example creates a mesh pattern using only VDD nets, then inserts vias between alternating intersections of layers M4 and M5 to form a checkerboard pattern of vias.

Full Example



```
create_pg_wire_pattern wire_base -layer @l -direction @d \
   -width @w -spacing @s -pitch @p -parameters {l d w p s}

create_pg_composite_pattern checker_board_vias -nets {VDD} \
   -add_patterns { \
      {{pattern: wire_base} {nets: VDD VDD} \
       {parameters: {M4 vertical 2 20 interleaving}} \
       {offset: 4}} \
      {{pattern: wire_base} {nets: VDD VDD} \
       {parameters: {M5 horizontal 2 20 interleaving}} \
       {offset: 4}}} \
   -via_rule { \
      {{{pattern_id: 1} {net_id: 1}} \
       {{pattern_id: 2} {net_id: 1}} {via_master: default}} \
      {{{pattern_id: 1} {net_id: 2}} \
       {{pattern_id: 2} {net_id: 2}} {via_master: default}} \
      {{{pattern_id: 1} {net_id: 2}} \
       {{pattern_id: 2} {net_id: 1}} {via_master: NIL}} \
      {{{pattern_id: 1} {net_id: 1}} \
       {{pattern_id: 2} {net_id: 2}} {via_master: NIL}} \
      {{intersection: undefined} {via_master: NIL}}}

set_pg_strategy core_strategy -core \
   -pattern {{name: checker_board_vias} {nets: VDD VSS } \
      {offset: {5 5}}}

compile_pg -strategies core_strategy
```

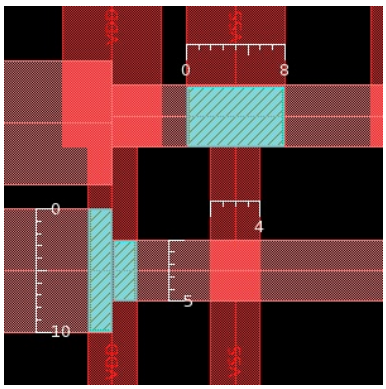
Commands used:
create_pg_wire_pattern, create_pg_composite_pattern, set_pg_strategy, and compile_pg.

Copy to script editor

Tapering Straps *Go to top*

This example creates a power structure with narrowed power straps at the center of the die. Vertical VDD and VSS straps are created on layer M6 and horizontal VDD and VSS straps are created on layer M7. The horizontal M7 straps have a width of 10 from the left die edge; starting at x=500 microns and ending at x=1000 microns the width narrows to 5 microns. The vertical M6 straps taper from 8 microns to 4 microns.

Full Example




```
create_pg_wire_pattern first_wire -layer @l -direction @d \
   -width @w -spacing @s -extend_low boundary \
   -high_end_reference_point @pt1 -parameters {l d w s pt1}

create_pg_wire_pattern center_wire -layer @l -direction @d \
   -width @w -spacing @s -low_end_reference_point @pt1 \
   -high_end_reference_point @pt2 -parameters {l d w s pt1 pt2}

create_pg_wire_pattern last_wire -layer @l -direction @d \
   -width @w -spacing @s -low_end_reference_point @pt2 \
   -extend_high boundary -parameters {l d w s pt2}
create_pg_composite_pattern tapering_base \
   -parameters {layer dir side_w center_w side_s center_s pt1 pt2} \
   -add_patterns { \
   {{pattern: first_wire} \
    {parameters: {@layer @dir @side_w @side_s @pt1}}} \
   {{pattern: center_wire} \
    {parameters: {@layer @dir @center_w @center_s @pt1 @pt2}}} \
   {{pattern: last_wire} \
    {parameters: {@layer @dir @side_w @side_s @pt2}}} \
}

create_pg_composite_pattern tapering_straps \
   -parameters {h_pt1 h_pt2 v_pt1 v_pt2} \
   -add_patterns { \
   {{pattern: tapering_base}
    {parameters: {M7 horizontal 10 5 2 7.5 @h_pt1 @h_pt2}} \
    {pitch: {0 30}}} \
   {{pattern: tapering_base}
    {parameters: {M6 vertical 8 4 2 6 @v_pt1 @v_pt2}} \
    {pitch: {25 0}}} \
}

set x_start 500
set x_end 1000
set y_start 500
set y_end 1000
set c_offset 5
set_pg_strategy core_strategy -core \
    -pattern {{name: tapering_straps} \
                  {nets: VDD VSS } \
                  {parameters: $x_start $x_end $y_start $y_end} \
                  {offset: 5 $c_offset}}

compile_pg -strategies core_strategy
```

Commands used:
create_pg_wire_pattern, create_pg_composite_pattern, set_pg_strategy, and compile_pg.

Copy to script editor